

UNIVERSIDAD ICESI

Facultad de Ingeniería, Diseño y Ciencias Aplicadas
Departamento de Computación y Sistemas Inteligentes

TAREA EVALUATIVA N.º 1

INTERPRETACIÓN DE PRINCIPIOS Y ASPECTOS CLAVE DE DISEÑO

Caso de estudio: plataforma ApexStore

Ingeniería de Software IV (09775 - TIC) – NRC 10865

Unidad 1: Diseño y Arquitectura de Software (SWEBOK V4, capítulos 2 y 3)

Profesores: Alejandro Muñoz Bravo y Alejandro Peñaranda Agudelo

Integrantes:

Danna Isabel Zambrano – Código: A00404131

Miguel Perez – Código: A00407054

Santiago de Cali, 2026-2

Contexto de ApexStore

ApexStore Technologies es una empresa de comercio electrónico que opera en cinco países. En eventos como CyberMonday o Black Friday el tráfico se multiplica por 100 y el checkout supera las 8.000 peticiones por segundo. El equipo de ingeniería propuso la arquitectura de la Figura 1, repartida en cuatro nodos físicos: clientes, backend, pasarelas de pago y base de datos.

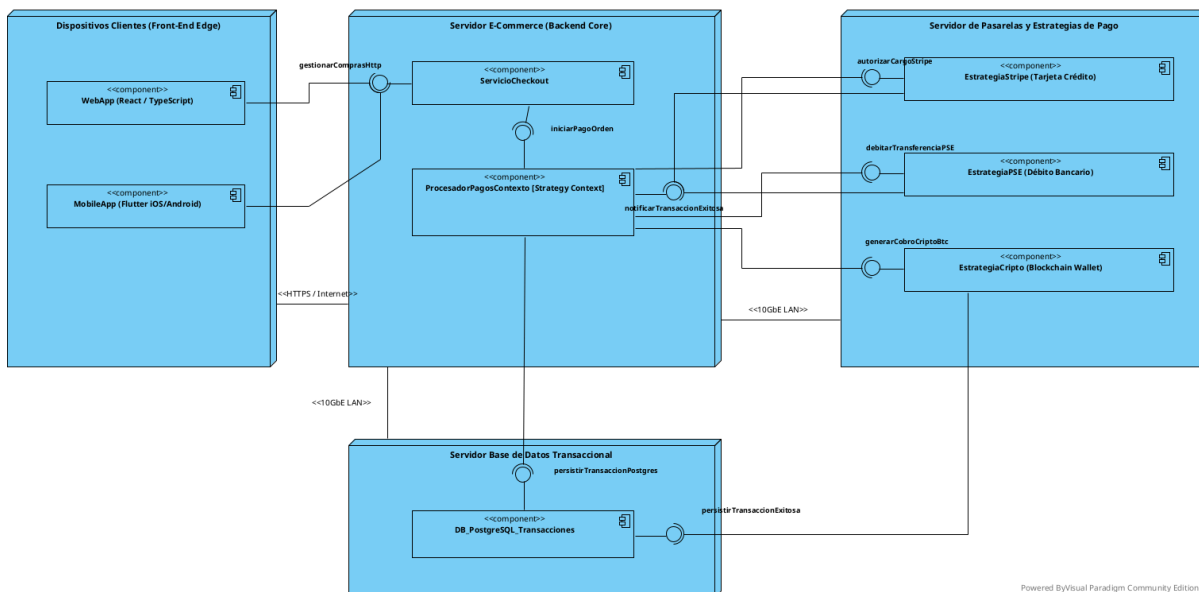


Figura 1. Diagrama de despliegue y componentes de ApexStore (enunciado).

El enunciado define cuatro Requerimientos Arquitectónicamente Significativos (RAS), que usamos como referencia en todo el informe:

- **RAS-01 (Disponibilidad y resiliencia en pagos):** el flujo de compra debe operar de forma asincrónica desacoplada; la lentitud o caída de pasarelas bancarias no debe congelar la atención de clientes (disponibilidad $\geq 99,9\%$).
- **RAS-02 (Rendimiento y latencia de red interna):** la orquestación interna de una transacción de pago entre el backend y los nodos especializados no debe superar 250 ms en el percentil 95.
- **RAS-03 (Consistencia e integridad ACID):** no se admiten pagos huérfanos, doble cobro ni inconsistencias contables; cada transacción se persiste atómica y auditadamente.
- **RAS-04 (Extensibilidad y mantenibilidad):** se deben poder agregar nuevos medios de pago con mínimo impacto, preservando el ocultamiento de información.

Punto 1: Principios de Diseño y Análisis Crítico

Para analizar el diagrama usamos los principios de diseño del SWEBOK V4 (capítulo 2) vistos en clase: abstracción, separación de preocupaciones, alta cohesión, bajo acoplamiento, ocultamiento de información, uniformidad, completitud y verificabilidad. Escogimos **dos buenos usos y tres malos usos**. En cada uno mostramos el recorte del diagrama del enunciado: **en verde lo que está bien y en rojo lo que está mal**.

1.1 Buen uso: fachada de compras gestionarComprasHttp

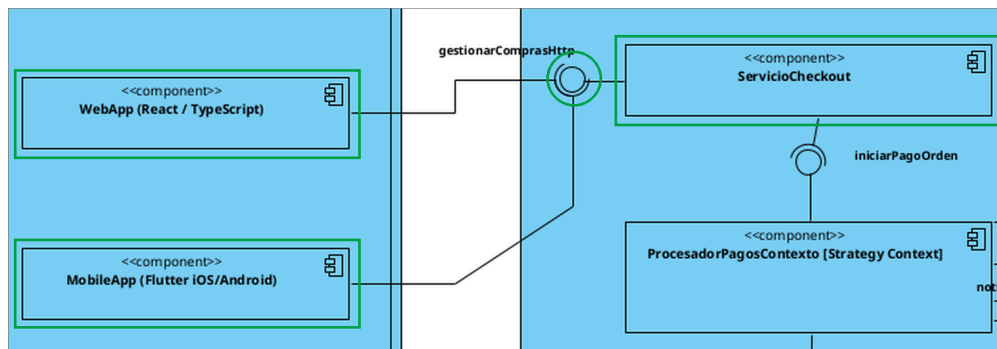


Figura 1.1. Los clientes solo se conectan a la interfaz de ServicioCheckout.

a) Evidencia en el diagrama	En la Figura 1.1, WebApp y MobileApp (nodo 1) tienen un solo enchufe, conectado a la bola gestionarComprasHttp que provee ServicioCheckout (nodo 2). Ningún cliente tiene líneas hacia ProcesadorPagosContexto, las estrategias o la base de datos. ServicioCheckout, a su vez, solo requiere iniciarPagoOrden.
Principio / clasificación	Ocultamiento de información, bajo acoplamiento y separación de preocupaciones. Buen uso.
b) Análisis del uso del principio	ServicioCheckout funciona como una fachada: los clientes conocen un solo contrato y no saben cómo se procesa el pago por dentro. Así la presentación (React y Flutter) queda separada de la lógica de negocio del backend. La notación también está bien, porque el ensamblaje une una interfaz provista con una requerida. El único detalle es que el nombre lleva el protocolo ("Http"), que es algo de transporte y no debería estar en el contrato.
c) Impacto en ApexStore	Se puede cambiar lo de adentro, como agregar un medio de pago o cambiar la base de datos, sin sacar otra versión de la app ni tocar la web, lo que apoya RAS-04. La lógica del checkout no se repite en dos lenguajes y solo el nodo 2 queda expuesto, lo que reduce la superficie de ataque. El costo es que ServicioCheckout se vuelve la única puerta de entrada y debe poder replicarse para no ser un cuello de botella en el pico de 8.000 peticiones/s.

1.2 Buen uso: cuatro nodos con la base de datos aparte

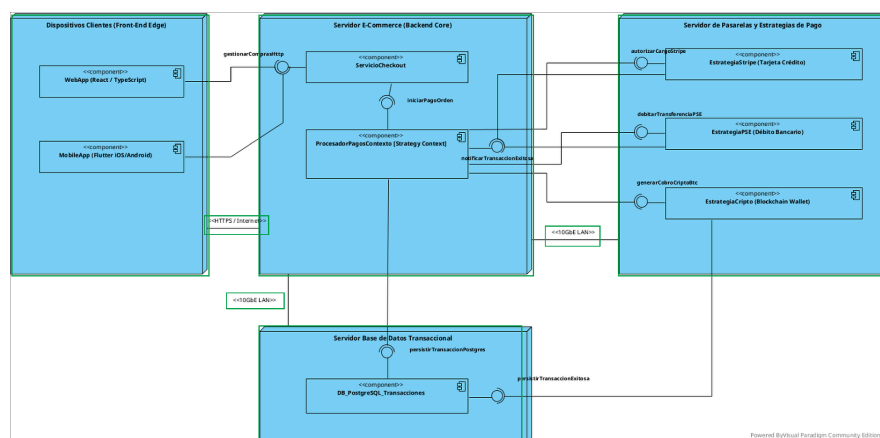


Figura 1.2. Los cuatro nodos y sus caminos de comunicación.

a) Evidencia en el diagrama	En la Figura 1.2 hay cuatro nodos (Dispositivos Clientes, Servidor E-Commerce, Servidor de Pasarelas y Servidor Base de Datos) y tres caminos de comunicación: nodo 1 con nodo 2 por «HTTPS / Internet», y nodo 2 con los nodos 3 y 4 por «10GbE LAN». La base de datos está sola en su nodo.
------------------------------------	---

Principio / clasificación	Modularidad, separación de preocupaciones y alta cohesión. Buen uso.
b) Análisis del uso del principio	Cada nodo agrupa componentes con un solo propósito: presentación, orquestación del pago, integración con terceros y persistencia. Es alta cohesión llevada al nivel físico. Además, solo el nodo 2 tiene camino hacia los clientes; los nodos 3 y 4 quedan detrás de él. El principio está bien aplicado, aunque la notación tiene detalles por corregir que revisamos en el punto 1.6.
c) Impacto en ApexStore	El nodo 2 se puede escalar por separado para el pico de tráfico, y si una pasarela se pone lenta el problema se queda en el nodo 3 sin tumbar la atención de clientes (RAS-01). La base de datos no compite por recursos con el backend, lo que ayuda con los 250 ms de RAS-02, y no es alcanzable desde Internet, como pide PCI-DSS. El costo es que cada llamada entre nodos pasa por la red; por eso importa que sea una LAN rápida de 10GbE.

1.3 Mal uso: no hay una interfaz Strategy común para las estrategias

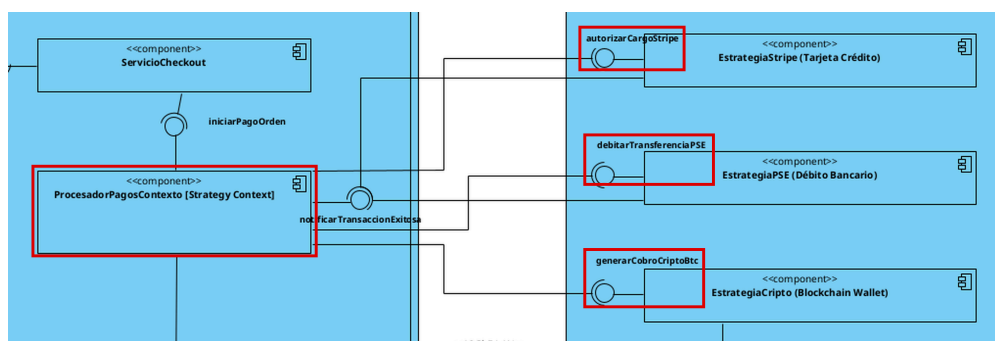


Figura 1.3. Cada estrategia expone una interfaz distinta y el contexto se conecta a las tres.

a) Evidencia en el diagrama	En el nodo 3 (Figura 1.3) cada estrategia provee una interfaz diferente: <code>autorizarCargoStripe(tokenTarjeta, montoUSD, cvcSeguridad)</code> , <code>debitarTransferenciaPSE(codigoBanco, tipoDoc, numCuenta, valorCOP)</code> y <code>generarCobroCriptoBtc(direccionWallet, redBlockchain, satoshis)</code> . <code>ProcesadorPagosContexto</code> , marcado como <code>Strategy Context</code> , tiene un enchufe distinto para cada una.
Principio / clasificación	Uniformidad, ocultamiento de información y bajo acoplamiento. Mal uso.
b) Análisis del uso del principio	En el patrón Strategy el contexto debería trabajar con una sola interfaz común que todas las estrategias cumplen, sin saber cuál está usando. Aquí pasa lo contrario: cada estrategia tiene su propio contrato, con nombre, datos y moneda diferentes (dólares, pesos y satoshis). El contexto depende de tres contratos concretos y tiene que conocer los detalles de cada medio, como el CVC de la tarjeta, el banco de PSE o la dirección de la wallet. Se rompe la uniformidad, porque tres piezas que hacen lo mismo (cobrar) se ven distinto, se rompe el ocultamiento de información y sube el acoplamiento. El componente se llama <code>Strategy Context</code> , pero en la práctica no aplica el patrón.
c) Impacto en ApexStore	Agregar un medio nuevo, por ejemplo una billetera digital, obliga a modificar <code>ProcesadorPagosContexto</code> : un enchufe nuevo, una interfaz nueva y otra rama en su lógica. Eso va en contra de RAS-04. Como todos los pagos pasan por el contexto, cada cambio en él pone en riesgo los medios que ya funcionan. Además, el CVC de la tarjeta termina viajando hasta el contexto, cuando no debería salir de la parte que maneja la tarjeta.

1.4 Mal uso: dependencia circular por notificarTransaccionExitosa

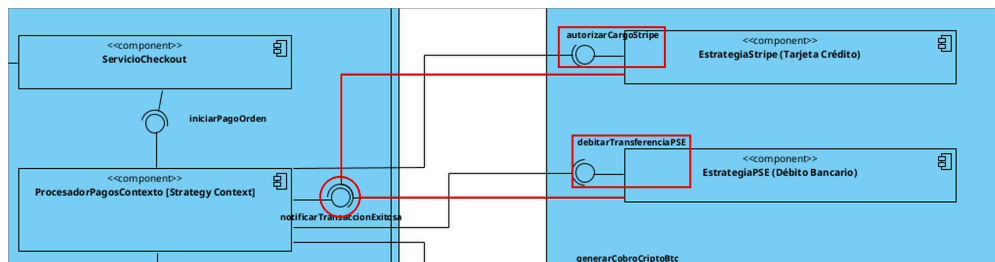


Figura 1.4. El contexto requiere a las estrategias y las estrategias requieren al contexto.

a) Evidencia en el diagrama	En la Figura 1.4, la interfaz provista notificarTransaccionExitosa está pegada a ProcesadorPagosContexto y sus enchufes vienen de EstrategiaStripe y EstrategiaPSE. Al mismo tiempo, el contexto requiere autorizarCargoStripe y debitarTransferenciaPSE, que proveen esas mismas estrategias.
Principio / clasificación	Bajo acoplamiento, verificabilidad y completitud. Mal uso.
b) Análisis del uso del principio	Usar un callback es buena idea, porque el contexto no se queda esperando al banco. Pero como está dibujado, el contexto requiere a las estrategias y las estrategias requieren al contexto: A requiere a B y B requiere a A. En UML esto es un error semántico, la dependencia circular fuerte, y hace que ninguno se pueda probar, reemplazar ni desplegar sin el otro. Además, el contrato está incompleto: el nombre solo cubre el éxito, no hay forma de avisar un rechazo y EstrategiaCripto ni siquiera lo usa.
c) Impacto en ApexStore	Cualquier cambio en el contexto obliga a revisar las estrategias y al revés, lo que perjudica la mantenibilidad (RAS-04). Un pago rechazado no tiene cómo reportarse, así que la orden puede quedarse pendiente indefinidamente, lo que afecta la integridad que pide RAS-03.

1.5 Mal uso: EstrategiaCripto escribe directamente en la base de datos

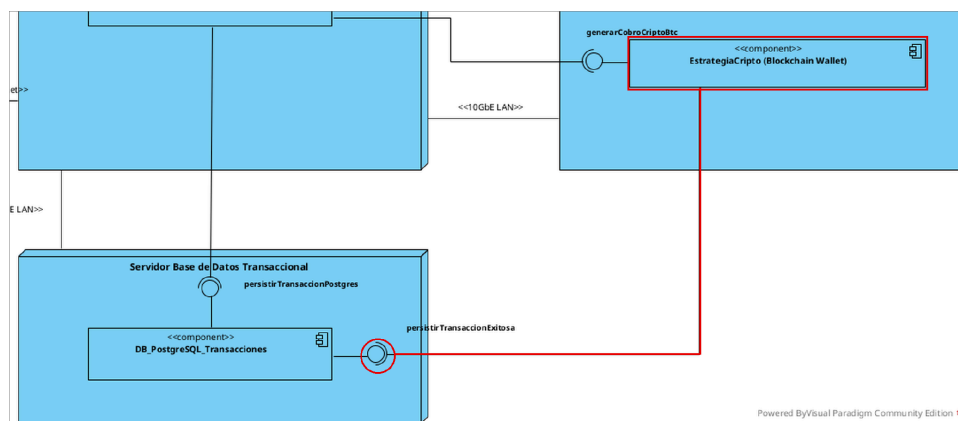


Figura 1.5. EstrategiaCripto conectada directamente a la base de datos.

a) Evidencia en el diagrama	En la Figura 1.5, EstrategiaCripto (nodo 3) tiene un enchufe conectado a persistirTransaccionExitosa, que provee DB_PostgreSQL_Transacciones (nodo 4). No usa notificarTransaccionExitosa como Stripe y PSE. La línea va del nodo 3 al nodo 4, pero el diagrama solo tiene caminos del nodo 2 al 3 y del nodo 2 al 4.
Principio / clasificación	Separación de preocupaciones, alta cohesión, ocultamiento de información y uniformidad. Mal uso.

b) Análisis del uso del principio	Una estrategia debería hacer una sola cosa: cobrar con su proveedor. EstrategiaCripto además registra el resultado, que le corresponde a quien maneja la orden. Queda un componente con baja cohesión y la persistencia repartida en dos lugares, con dos interfaces distintas sobre los mismos datos. Cripto se salta al componente dueño de la transacción y llega directo a los datos, lo que viola el encapsulamiento. Tampoco hay uniformidad: dos estrategias reportan por callback y la tercera de otra forma, lo que empeora el problema del punto 1.3. Y hay un error semántico de despliegue: dos nodos que se comunican sin un camino entre ellos.
c) Impacto en ApexStore	El contexto no se entera del pago con cripto: la orden puede quedar pendiente con el dinero ya recibido o registrarse dos veces, justo lo que prohíbe RAS-03. Y para que esto funcionara, el nodo 3 necesitaría credenciales de escritura en la base de datos y una conexión directa al nodo 4, lo que rompe la separación de la red.

1.6 Errores e imprecisiones de notación

Además de los cinco puntos, revisamos el diagrama como una especificación (ISO/IEC 19505). algunas cosas que aparecen UML sí las permite, así que no las llamamos errores de sintaxis sino imprecisiones: el diagrama no está mal escrito, pero queda incompleto para quien lo tenga que desplegar. Otras sí son errores semánticos, porque describen algo que no puede pasar en la realidad. Todo se puede ver en la Figura 1.

Imprecisiones. Los nodos no dicen si son «device» o «executionEnvironment», así que no se sabe qué es hardware y qué es el software que ejecuta el código. No aparece ningún entorno de ejecución (el navegador de la WebApp, la JVM del backend y de las pasarelas, el motor de PostgreSQL). Los componentes no tienen un «artifact» que los manifieste, que es lo que realmente se despliega. Y la base de datos está dibujada como «component», mezclando el componente lógico y la base de datos desplegada.

Errores semánticos. «Dispositivos Clientes» mete en un solo nodo un computador y un smartphone, que son dos equipos distintos. El camino «10GbE LAN» describe el medio físico pero no el protocolo con el que se comunican los nodos. Y el nodo 3 no tiene ningún camino hacia Stripe, el banco o la red blockchain, que es con quienes tiene que hablar para cobrar.

Lo que sí está bien. Los nodos son cubos, los componentes usan «component» con su ícono, todos los ensamblajes unen una interfaz provista con una requerida y los caminos de comunicación son líneas sin flecha.

Punto 2: Análisis Profundo de los Aspectos Clave de Diseño

2.1 Dimensión 1: Interacción y Presentación

¿Por qué los clientes front-end interactúan exclusivamente con la interfaz gestionarComprasHttp provista por ServicioCheckout, en lugar de conectarse directamente con el procesador de pagos o la base de datos?

R// Porque así cada parte hace solo lo suyo WebApp y MobileApp muestran información y capturan lo que el usuario quiere hacer validar la compra, crear la orden, elegir la estrategia y registrar la transacción le toca al backend. Si los clientes llamarán directo al ProcesadorPagosContexto, esa lógica tendría que repetirse y cualquier diferencia entre las dos versiones terminaría en un error contable. También es ocultamiento de información el procesador, las estrategias y PostgreSQL son detalles internos, y con la fachada el backend los puede cambiar sin obligar a actualizar la app móvil, que depende de las tiendas de aplicaciones y de que el usuario la actualice.

Además está la seguridad los clientes corren en equipos que ApexStore no controla y llegan por Internet. Conectarlos a la base de datos obligaría a meter credenciales dentro de la app, donde se pueden extraer, y a exponer el nodo 4. Con la fachada, lo único expuesto es una interfaz HTTPS donde se autentica al usuario y se valida cada solicitud y en un pico de 8.000 peticiones/s ese único punto de entrada permite limitar peticiones y replicarse sin tocar lo demás. El precio es que si ServicioCheckout falla, nadie puede comprar por eso debe ser liviano (recibir, validar y delegar) y replicable.

2.2 Dimensión 2: Concurrencia y Rendimiento

En un pico de 8.000 peticiones/s, ¿por qué la combinación de despacho síncrono inicial con confirmación asíncrona vía webhook/callback protege el thread pool del servidor frente a retardos bancarios de 15 segundos?

R// La afirmación es correcta, siempre que el despacho síncrono solo entregue el cobro a la pasarela y espere un recibido y que la respuesta del banco llegue después por el callback.

El servidor E-Commerce atiende cada petición con un hilo de un thread pool de tamaño fijo, y mientras un hilo espera no puede atender a nadie más. Si cada compra dejará su hilo esperando los 15 segundos del banco en ese tiempo seguirían llegando miles de compras por segundo sin que se libere ningún hilo. El pool se agotaría casi de inmediato y el servidor dejaría de responder, incluso a los pagos con medios que sí funcionan.

El despacho síncrono corto evita eso el contexto le entrega el cobro a la estrategia del nodo 3 por la LAN y solo espera el acuse de recibido, que no depende del banco con ese acuse le responde al cliente que la orden quedó pendiente y el hilo vuelve al pool. Cuando el banco termina, la pasarela avisa por el callback (notificarTransaccionExitosa). Eso es el desacoplamiento temporal de la gestión de eventos quien emite el evento no se queda esperando, y el aviso ocupa un hilo solo lo que tarda en actualizar la orden. Así la latencia que siente el cliente es la del despacho y no la del banco, lo que hace posible cumplir los 250 ms de y un banco lento no le quita hilos a las demás compras.

En el diagrama del enunciado esto se cumple solo en parte. No se aclara qué espera la llamada síncrona ni se define un tiempo máximo. Cripto no usa el callback (punto 1.5), y el callback solo cubre el éxito, así que un rechazo deja la orden pendiente. El costo del esquema es que el cliente no recibe la respuesta final de una vez y el sistema debe manejar estados intermedios y avisos repetidos o tardíos. Aun así vale la pena, porque esperar al banco tumbaría el servidor en el pico.

2.3 Dimensión 3: Tolerancia a Fallos y Manejo de Errores

Si la pasarela PSE no responde o la red blockchain se congestiona, ¿cómo debe reaccionar el contexto para que el fallo de una estrategia no colapse las transacciones de los otros métodos de pago?

R// Lo primero es aislar el fallo cada estrategia debe tener su propio tiempo máximo de espera y su propio circuit breaker, un interruptor que si un medio falla varias veces seguidas deja de enviarle cobros por un rato y responde de una vez que no está disponible. Así un problema en PSE no consume los recursos de Stripe o Cripto, y al cliente se le puede ofrecer otro medio.

También hay que tener en cuenta cómo funciona cada medio. En PSE el cliente va a la página de su banco a aprobar el débito, y puede demorarse, abandonar o el banco puede no responder. En blockchain, con la red congestionada, las confirmaciones pueden tardar minutos.

En los dos casos el pago no ha fallado, simplemente no se sabe todavía. Por eso el contexto no debe marcarlo fallido por un timeout corto, sino dejarlo en verificación hasta que llegue el resultado real, y ahí pasa a Aprobado o Rechazado así se evita dar por fallido un pago que el banco sí cobró.

En el manejo de excepciones, todo error de comunicación se convierte en un estado guardado y auditado, nunca en una orden perdida. Solo se reintenta lo que se puede repetir sin efecto, como la entrega del callback el cobro no se reintenta solo y cada orden lleva una clave de idempotencia si el cliente vuelve a pagar porque PSE no responde, el sistema ve que esa orden ya tiene un pago en curso y no cobra dos veces. El costo es que algunos clientes verán su orden en verificación un rato, pero es mejor que perder un pago o cobrar doble.

2.4 Dimensión 4: Gestión de Eventos y Control de Flujo

Compare el flujo de control entre el callback que recibe ProcesadorPagosContexto y la inserción directa que hace EstrategiaCripto. Si Cripto escribe directo a la base de datos, ¿cómo se entera el contexto para actualizar la orden y enviar la confirmación al cliente?

R// Con Stripe y PSE, el contexto despacha el cobro, la pasarela lo procesa y la estrategia invoca `notificarTransaccionExitosa` el contexto recibe ese evento y actualiza la orden con `persistirTransaccionPostgres`. El contexto controla el flujo de principio a fin. Con Cripto el flujo se rompe al final EstrategiaCripto llama `persistirTransaccionExitosa`, que va directo a la base de datos y el contexto no recibe nada.

Entonces con el diagrama como está, el contexto no se entera. PostgreSQL tiene un mecanismo para avisar cambios (`Listen/Notify`), pero no está en el diseño y usarlo amarraría el contexto a la base de datos la otra opción, consultar la base cada cierto tiempo tampoco está y genera carga y retraso mientras tanto nadie actualiza la orden de forma controlada o se queda pendiente para siempre o Cripto tendría que conocer también el modelo de órdenes.

Además, la confirmación al cliente no está resuelta para ningún medio, ni siquiera para Stripe o PSE, `ServicioCheckout` solo requiere `iniciarPagoOrden` y no hay interfaz para que le avisen ni para que el cliente consulte su orden. El problema de fondo es que hay dos flujos de control para el mismo evento el pago terminó y en uno el contexto queda por fuera. Eso lleva a pagos huérfanos o duplicados y rompe la uniformidad del Strategy.

2.5 Dimensión 5: Persistencia de Datos e Integridad Transaccional

¿Por qué es imperativo que DB PostgreSQL Transacciones garantice propiedades ACID para los cobros, en contraste con un modelo de consistencia eventual (BASE)?

R// Porque en un cobro no se puede aceptar ni un momento de datos inconsistentes. Con la atomicidad, registrar el pago y pasar la orden a Aprobado es un solo paso si el servidor se cae en la mitad, no queda un pago sin orden aprobada ni una orden aprobada sin pago. Con la consistencia, cada cambio respeta las reglas clave de idempotencia única para que un reintento no cree otro cobro, todo pago ligado a una orden, montos positivos y estados que solo avanzan (aprobado no vuelve a pendiente). El aislamiento importa a 8.000 peticiones/s porque pueden llegar a la vez el callback y un reintento del cliente sobre la misma orden sin él, ambos leerían pendiente y aplicarían el resultado dos veces y la durabilidad garantiza que un pago ya confirmado siga ahí aunque el nodo 4 se reinicie, lo que en PostgreSQL da el registro de escritura anticipada.

BASE acepta que por un tiempo distintas copias de los datos muestren cosas diferentes, a cambio de más disponibilidad y escalabilidad. Sirve en ApexStore para el catálogo, las reseñas o el buscador, donde un dato viejo por unos segundos no hace daño. En el núcleo contable no si dos copias no coinciden sobre si un pago existe, se puede despachar sin cobrar, cobrar dos veces o no cuadrar con el banco y el dinero que ya se movió no se devuelve solo. RAS-03 lo exige bajo ninguna circunstancia. El precio de ACID es que limita la escalabilidad por los bloqueos, así que se usa solo donde es indispensable la relación con el banco sí es eventualmente consistente, porque el resultado llega después y justamente por eso el registro interno tiene que ser estricto.

2.6 Dimensión 6: Distribución de Componentes

Justifique la topología física en 4 nodos de ejecución interconectados por HTTPS / Internet y 10GbE LAN bajo el estándar de seguridad de datos financieros (PCI-DSS).

R// Cada nodo tiene una razón de ser el nodo 1 son equipos del usuario en los que no se puede confiar, así que solo presentan y capturan datos. El nodo 2 es la única puerta pública, concentra las reglas de negocio y se escala aparte para el pico. El nodo 3 aísla a los terceros financieros si una pasarela se pone lenta el daño se queda ahí (RAS-01) y se limita qué equipos tocan datos de pago. El nodo 4 es el activo más crítico dedicado, no compite por CPU y disco, se protege y respalda aparte y queda fuera de Internet.

Entre el nodo 1 y el 2 la comunicación pasa por Internet, por eso se usa HTTPS, que cifra, protege la integridad y autentica al servidor. Entre los nodos 2, 3 y 4 va por una LAN privada de 10 Gb/s, rápida (RAS-02) y, sobre todo, segmentada el nodo 4 no tiene acceso desde ni hacia Internet y el nodo 3 no acepta conexiones de afuera pero sí tiene salida controlada hacia Stripe, PSE y la red blockchain.

Esto encaja con PCI-DSS. El requisito 1 pide controles de seguridad de red, y separar la zona de datos de pago reduce el alcance del entorno de tarjetahabientes. El requisito 3 prohíbe guardar el CVC después de autorizar, por eso conviene tokenizar la tarjeta en el cliente y no pasar cvcSeguridad por el contexto como en el diagrama original. El requisito 4 pide cifrar los datos por redes públicas, lo que cubre HTTPS, y el requisito 10 pide registrar los accesos, lo que coincide con la persistencia auditada de RAS-03.

El costo de cuatro nodos es que cada pago hace varios saltos de red y hay más puntos que pueden fallar la LAN rápida reduce ese costo. Además, un solo nodo de base de datos y uno de pasarelas son puntos únicos de falla frente al 99,9 % de RAS-01, así que convendría una réplica de la base de datos con failover y más de una instancia del nodo de pasarelas.

Punto 3: Rediseño Arquitectónico del Patrón Strategy y Flujo Asíncrono

Con los malos usos del Punto 1 y las imprecisiones del punto 1.6 rediseñamos el diagrama en Visual Paradigm (Figura 3.1). La idea fue conservar lo que ApexStore ya hacía bien (los cuatro nodos, la fachada y el callback asíncrono) y cambiar la forma en que se conectan los componentes del pago.

3.1 Qué se corrigió y por qué

El cambio más importante está en el patrón Strategy (punto 1.3). Ahora existe una sola interfaz, `IEstrategiaPago`, que proveen las cuatro estrategias y que requiere `ProcesadorPagosContexto`. Los datos propios de cada medio (el token de la tarjeta, el banco de PSE o la wallet) viajan como un dato que solo la estrategia sabe leer, así que el contexto ya no conoce los detalles de ningún medio. Con esto se recuperan la uniformidad y el ocultamiento de información que pide RAS-04.

Para romper la dependencia circular (punto 1.4) el callback ya no lo provee el contexto sino un componente aparte, `ReceptorResultadosPagos`, por medio de `INotificacionPago`. Las dependencias quedaron en un solo sentido el contexto requiere a las estrategias, las estrategias requieren al receptor y el receptor requiere a la persistencia, sin que nada se devuelva. Así cada componente se puede probar y reemplazar por separado. Además, esa interfaz ya no avisa solo el éxito informa si el pago quedó aprobado, rechazado o en verificación, lo que completa el contrato.

`EstrategiaCripto` dejó de escribir en la base de datos (punto 1.5). Ahora reporta por el mismo callback que las demás y ninguna estrategia conoce la persistencia, por eso en el diagrama ya no hay camino entre el nodo 3 y el nodo 4. La única entrada a los datos es `IPersistenciaTransaccional`, que provee `ServidorPersistencia`. También resolvimos lo que vimos en el punto 2.4: `IGestionCompras` permite consultar el estado de la orden, así que el cliente siempre puede saber cómo terminó su pago.

Por último, corregimos la notación del punto 1.6. Cada nodo dice si es «device» o «executionEnvironment», los componentes se despliegan en artefactos con «manifest» (por ejemplo `backend.jar` y `pagos.jar`), el computador y el smartphone son nodos separados y los caminos indican el protocolo (HTTPS e ICE/TCP). También aparece el nodo de los proveedores de pago externos, al que el nodo 3 sale por HTTPS, y la base de datos quedó como el DBMS PostgreSQL con el artefacto `apexstore_tx`, al que accede el componente `ServidorPersistencia`.

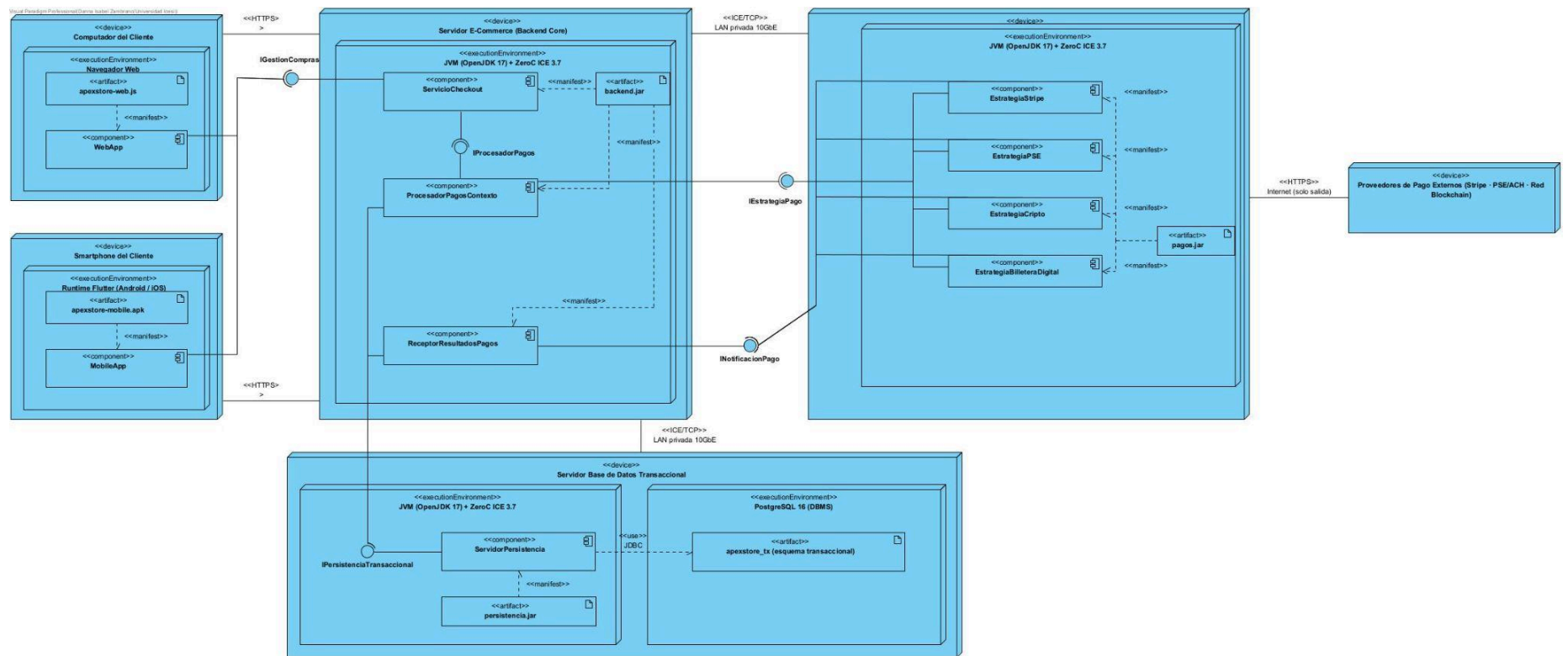


Figura 3.1. Diagrama corregido.

3.2 Cómo quedan conectados los componentes

En el diagrama corregido cada interfaz tiene un solo dueño y queda claro quién la usa. ServicioCheckout provee IGestionCompras, que requieren WebApp y MobileApp; ProcesadorPagosContexto provee IProcesadorPagos, que requiere ServicioCheckout las cuatro estrategias proveen IEstrategiaPago, que requiere el contexto ReceptorResultadosPagos provee INotificacionPago, que requieren las estrategias, y ServidorPersistencia provee IPersistenciaTransaccional, que requieren el contexto y el receptor. Todos los ensamblajes unen una interfaz provista con una requerida.

Con esto, todos los medios de pago siguen el mismo camino. El cliente envía la compra a ServicioCheckout, el contexto registra la orden como pendiente antes de cobrar y la despacha a la estrategia del medio por IEstrategiaPago, que responde de inmediato con un acuse. Cuando la pasarela externa contesta, la estrategia avisa al receptor por INotificacionPago y este guarda el resultado. El cliente consulta su orden hasta ver el estado final. Como el contexto ya no sabe qué medio está usando, agregar uno nuevo es agregar una estrategia así entró EstrategiaBilleteraDigital sin tocar el resto del diseño (RAS-04).

Lo que no se resolvió en el diagrama es la redundancia sigue habiendo un solo nodo de base de datos, que es un punto único de falla (punto 2.6). El backend no guarda estado propio, así que sí se puede replicar, pero la réplica de la base de datos quedó fuera del alcance de esta tarea.

Punto 4: Mapeo hacia el Diseño Detallado y Código Java (RA4)

Implementamos el diagrama corregido en Java 17 con ZeroC ICE 3.7. El proyecto completo va adjunto (Taller1_Software4) con un README que explica cómo compilarlo y ejecutarlo. Cada nodo del diagrama es un programa independiente y cada artefacto es el jar que se instala en ese nodo. Los nodos solo se conocen por las interfaces ICE, y sus direcciones se configuran en config/, así que se pueden repartir en varios equipos sin recompilar.

El cliente (cliente.jar, clase ClienteApp) simula a WebApp y MobileApp y solo conoce la fachada del backend. El Servidor E-Commerce (backend.jar, puerto 10002) publica ServicioCheckout, ProcesadorPagosContexto y ReceptorResultadosPagos, y entre ellos también se hablan por proxies ICE, de modo que cada uno depende de la interfaz del otro y no de su clase. El Servidor de Pasarelas (pagos.jar, puerto 10001) publica las cuatro estrategias, y el Servidor Base de Datos (persistencia.jar, puerto 10000) publica ServidorPersistencia, que guarda las transacciones por JDBC en el esquema apexstore_tx.

4.1 Contrato ICE

Las cinco interfaces provistas del diagrama son exactamente las cinco interfaces del archivo Slice. El monto va en la unidad mínima de la moneda para evitar errores de redondeo, y idOrden sirve como clave de idempotencia. Además de las interfaces, el contrato define los datos de la orden y del resultado, y los cuatro estados del pago: Pendiente, EnVerificacion, Aprobado y Rechazado.

4.2 Cómo se ve cada corrección en el código

Strategy con una sola interfaz las cuatro estrategias heredan de EstrategiaPagoBase, que implementa IEstrategiaPago. Cada una solo define cómo validar sus datos y cómo cobrar. El contexto no tiene un if por medio de pago busca la estrategia por configuración.

Para agregar la billetera digital bastó con crear su clase, registrarla en ServidorPasarelas y agregar la línea Pagos.billetera.Proxy=EstrategiaBilleteraDigital:tcp -h 127.0.0.1 -p 10001. El contexto, las otras estrategias y el contrato no cambiaron (RAS-04).

Acuse inmediato y resultado por callback. pagar solo valida los datos, deja el cobro en el pool de hilos propio de esa estrategia y responde Pendiente. El resultado llega después al receptor, y si la notificación falla se reintenta hasta 3 veces, lo cual es seguro porque aplicarla dos veces no cambia nada. Como cada estrategia tiene su propio pool, una pasarela lenta solo llena su propia cola (RAS-01).

Tolerancia a fallos. El contexto llama a pagar con un tiempo máximo de 2 s. Si no hay conexión con la pasarela, rechaza la orden, porque el cobro nunca llegó. Si la llamada se vence, la deja EnVerificacion y no como fallida, porque la pasarela pudo haber cobrado.

Integridad ACID. La orden se registra antes de cobrar, así nunca existe un cobro sin orden id_orden es la llave primaria, entonces una orden repetida no se registra ni se cobra otra vez cada cambio de estado es una transacción JDBC que bloquea la fila con SELECT ... FOR UPDATE, revisa que la transición sea válida un pago aprobado no vuelve a pendiente y escribe en la tabla auditoria en la misma transacción. Los datos del medio de pago (tokens, cuentas, wallets) nunca se guardan.

4.3 Ejecución

El README del proyecto (Figura 4.1) explica dos formas de ejecutarlo la opción A corre toda la demo en un solo equipo y la opción B levanta cada nodo del diagrama corregido en su propia terminal, en el orden persistencia, pagos, backend y cliente. Probamos las dos. En ambas las pasarelas son simuladas y no se conectan a ningún servicio financiero real.



Figura 4.1. Instrucciones de ejecución del README del proyecto.

Opción A: demo completa en un equipo. Compilamos con gradle assemble y luego corrimos scripts/demo-local.sh, que levanta los tres servidores, ejecuta los ocho escenarios del cliente y apaga los servidores al final (Figura 4.2). Cada escenario muestra la respuesta inmediata del

backend con su tiempo y las consultas posteriores del cliente hasta ver el estado final. Los montos se imprimen en la unidad mínima de cada moneda, como define el contrato Slice.

```

~/Documents/ReposGit/Taller1_Software4 > on P main ?1
) gradle assemble

BUILD SUCCESSFUL in 522ms
7 actionable tasks: 7 up-to-date

~/Documents/ReposGit/Taller1_Software4 > on P main ?1
) scripts/demo-local.sh

== stripe: stripe por 12000 USD ==
respuesta (46 ms) Pendiente STRIPE-RECIBIDO Pago en proceso
consulta Aprobado STRIPE-APROBADO Cargo autorizado

== pse: pse por 45000000 COP ==
respuesta (5 ms) Pendiente PSE-RECIBIDO Pago en proceso
consulta Pendiente REGISTRADA Orden recibida
consulta EnVerificacion PSE-VERIFICANDO El banco aún no confirma el débito
consulta Aprobado PSE-APROBADO El banco confirmó el débito

== cripto: cripto por 250000 BTC ==
respuesta (5 ms) Pendiente CRIPTO-RECIBIDO Pago en proceso
consulta Pendiente REGISTRADA Orden recibida
consulta Aprobado CRIPTO-APROBADO Transacción confirmada en la red

== billetera: billetera por 7500 USD ==
respuesta (5 ms) Pendiente BILLETERA-RECIBIDO Pago en proceso
consulta Aprobado BILLETERA-APROBADO Pago autorizado desde la billetera

== rechazo: stripe por 9900 USD ==
respuesta (4 ms) Pendiente STRIPE-RECIBIDO Pago en proceso
consulta Rechazado STRIPE-DECLINADA El emisor declinó la tarjeta

== invalido: stripe por 9900 USD ==
respuesta (7 ms) Rechazado STRIPE-DATOS Stripe solo recibe tarjetas tokenizadas (tok...)

== desconocido: paypal por 5000 USD ==
respuesta (1 ms) Rechazado MEDIO-NO-SOPORTADO No hay una estrategia configurada para el medio paypal

== duplicada: billetera por 3000 USD ==
respuesta (3 ms) Pendiente BILLETERA-RECIBIDO Pago en proceso
reenvio Pendiente REGISTRADA Orden recibida
consulta Aprobado BILLETERA-APROBADO Pago autorizado desde la billetera

Registros de los servidores en build/logs/

```

Figura 4.2. Salida del cliente al ejecutar scripts/demo-local.sh.

En los pagos que salen bien (stripe, cripto y billetera) el cliente recibe Pendiente en milisegundos y al consultar la orden ya está Aprobado. Esto pasa porque el backend solo espera el acuse de la estrategia y el resultado llega después por el callback (RAS-01, RAS-02). La primera consulta de pse y cripto devuelve registrada, porque la orden se guarda antes de cobrar (RAS-03). La billetera digital sigue el mismo recorrido que los demás medios sin que se haya modificado el contexto (RAS-04).

El escenario pse muestra el estado intermedio mientras el banco no confirma, la orden queda en EnVerificacion y solo después pasa a Aprobado, en lugar de marcarse como fallida. En rechazo, el emisor declina la tarjeta y la orden termina en Rechazado con STRIPE-DECLINADA, lo que muestra que el callback también informa los rechazos (corrige el punto 1.4).

Los últimos tres escenarios prueban las validaciones. En invalido, Stripe rechaza en 7 ms una tarjeta sin tokenizar (STRIPE-DATOS) solo la estrategia lee los datos de pago y el contexto nunca ve la tarjeta. En desconocido, paypal se rechaza en 1 ms con MEDIO-NO-SOPORTADO, porque el contexto busca la estrategia por configuración y no hay ninguna para ese medio. En duplicada, el reenvío de la misma orden devuelve registrada y la consulta muestra un solo Aprobado: idOrden funciona como clave de idempotencia y no hay doble cobro (RAS-03).

Opción B: una terminal por nodo. Para ver la distribución del diagrama corregido, levantamos cada nodo en su propia terminal (Figura 4.3). Arriba a la izquierda está el Nodo 4 (persistencia): registra cada orden antes del cobro y cada cambio de estado, y cuando llega la orden duplicada escribe que ORD-6921C40B ya estaba registrada, se ignora. Su primera línea muestra la conexión JDBC a H2 en modo PostgreSQL. Arriba a la derecha está el Nodo 3 (pasarelas) cada estrategia recibe el cobro, lo deja en proceso y después notifica el resultado al receptor la orden ORD-42A19DEA se rechaza ahí mismo porque la tarjeta no venía tokenizada. Abajo a la izquierda está el Nodo 2 (backend), donde se ve el recorrido checkout, contexto y receptor de cada orden, y abajo a la derecha el Nodo 1 (cliente), con la misma salida de la Figura 4.2.

The figure shows four terminal windows running the 'Taller1_Software4' application. The top-left window (Nodo 4) shows a list of orders and their states, such as 'ORD-AE20A2B0 registrada (stripe, 12000 USD)' and 'ORD-6921C40B ya estaba registrada, se ignora'. The top-right window (Nodo 3) shows the processing of orders by different payment strategies, including 'ORD-AE20A2B0 recibida, en proceso' and 'ORD-42A19DEA rechazada: Stripe solo recibe tarjetas tokenizadas (tok...)'. The bottom-left window (Nodo 2) shows the checkout process, including 'orden ORD-42A19DEA de cliente-0042 por 9900 USD' and 'orden ORD-6921C40B de cliente-0042 por 3000 USD'. The bottom-right window (Nodo 1) shows the client's interaction, including 'respuesta (7 ms) Rechazado STRIPE-DATOS' and 'respuesta (1 ms) Rechazado MEDIO-NO-SOPORTADO'.

Figura 4.3. Ejecución con una terminal por nodo: persistencia (Nodo 4), pasarelas (Nodo 3), backend (Nodo 2) y cliente (Nodo 1).

Las marcas de tiempo de la Figura 4.3 confirman el desacoplamiento. El pago PSE ORD-CADD9798 llega al Nodo 3 a las 15:21:14.308, pasa a EnVerificacion a las 15:21:15.671 y se aprueba a las 15:21:18.677, más de cuatro segundos después, mientras el cliente ya había recibido su acuse en 5 ms. La orden con paypal (ORD-F1910DE6) aparece en el checkout pero nunca llega al Nodo 4, porque el contexto la rechaza antes de registrarla. Y ninguna estrategia del Nodo 3 escribe en la base de datos: todo cambio de estado en el Nodo 4 llega por el backend. Estas mediciones son locales y no son una prueba de carga de 8.000 peticiones/s, pero muestran que la respuesta al cliente no depende del banco.

Diferencias con el diagrama. El cliente de prueba usa ICE sobre TCP en lugar de HTTPS porque es una aplicación Java; en producción ese camino iría cifrado. Por defecto la base de datos es H2 en memoria en modo PostgreSQL para que la demo no exija instalar nada, y con BaseDatos.Ur1 se apunta a un PostgreSQL real sin cambiar código.

Conclusión

El diagrama original tenía buenas ideas la separación en nodos y el callback, pero el patrón Strategy estaba aplicado solo de nombre tres contratos distintos, un ciclo de dependencias y una estrategia escribiendo directo en la base de datos. Al corregirlo con una interfaz común, un

receptor aparte para los resultados y una sola puerta hacia la persistencia, el diseño quedó más fácil de extender y de probar, y el código lo confirma. Agregar la billetera digital no tocó el contexto y los fallos de una pasarela no afectan a las demás. Lo más importante que nos llevamos es que los principios de diseño no son teoría suelta cada uno se traduce en una decisión concreta que se nota cuando el sistema está bajo presión.

Anexo: Bitácora de Uso de IAG

1) Herramienta y versión. Claude (Anthropic), modelo Claude Haiku 4.5. El borrador de los Puntos 1 y 2, el diagrama corregido en Visual Paradigm y el código Java con ICE los hicimos nosotros. Usamos la IA como apoyo para revisar ese trabajo corregir ortografía y redacción del borrador siguiendo unas notas de revisión, revisar el informe contra la rúbrica, comprobar si nuestro diagrama corregido resolvía los errores del Punto 1 y ayudarnos a redactar la explicación de los Puntos 3 y 4 a partir de nuestro diagrama y nuestro código. Durante la implementación también consultamos DeepSeek-V3.1 y Claude Haiku 4.5 como revisores del código.

2) Prompts principales. Antes de cada prompt explicamos qué buscábamos el texto entre comillas está transcrito tal como lo escribimos.

Prompt 1: Le pedimos revisar el punto 1 y 2 de acuerdo a los errores semánticos, sintácticos, léxicos de los diagramas UML, ya que de pronto podíamos omitir algunos errores puntuales con eso determinamos los principios de diseño de una mejor forma le mandamos la imagen del diagrama del enunciado se le escribió "De acuerdo a este diagrama de despliegue identifique errores como que ningún device tiene un entorno de ejecución como el executionEnvironment, también que hay 3 interfaces diferentes para lo de las estrategias y una conexión circular entre la clase ProcesadorPagosContexto y EstrategiaStripe y estrategia PSE pero de acuerdo a los principios de diseño esos errores identificados estarían bien o omito más errores que de pronto no los este visualizando en el diagrama"

Prompt 2. Le pasamos nuestro código ICE, el diagrama corregido y el documento para que nos ayudara a redactar el informe y a revisar que todo cumpliera la rúbrica sin pasarse de extensión: "Para identificar mejor los errores del punto uno necesito que me señales con líneas verdes lo que esta bien y con líneas rojas lo que esta mal te mando aquí las capturas para que me señales bien los errores, necesito mejor la redacción en los demás puntos y portada del informe tipo APA acomodar mejor las imágenes para dar mas orden al documento"

Prompt 3 :le preguntamos sobre si las conexiones del diagrama que estamos corrigiendo si estaban de acuerdo a los principios de diseño, si de pronto estábamos omitiendo algo que nos indicara cual y el porque para mayor comprensión.

Prompt 4: Se le pidió ayuda con la organización del informe separación clara por punto "necesito organizar mejor la parte 2 y 4 del informe porque de pronto redacte algunas cosas mal, revisa que siga la rúbrica, ayúdame a organizar el documento"

3) Evaluación crítica y corrección humana. Nuestra forma de trabajar fue mostrarle lo que teníamos y preguntarle qué mejorar según la rúbrica. Sus sugerencias nos ayudaron, pero al

revisarlas contra los conceptos vistos en clase encontramos varias falencias que tuvimos que corregir:

- Sugirió justificar la pregunta 2.2 con la ley o formula que no se vio en clase. La descartamos porque se le pregunto al profesor e indicó que no se debían usar fórmulas, sino justificar con los aspectos clave de diseño vistos en clase (conurrencia, gestión de eventos y tolerancia a fallos).
- Al revisar nuestro primer diagrama corregido nos señaló lo que faltaba (la interfaz Strategy común, un callback que cubriera rechazos y los «manifest» de pagos.jar). Esas correcciones las hicimos nosotros en Visual Paradigm (Figura 3.1).
- El código del Punto 4 y el diagrama corregido son nuestros la IA los usó como base para ayudarnos a organizar la redacción de los textos.